

Настройка CORS в трёх веб приложениях

Отчёт по практической работе

Выполнил:

Аньшин Сергей

Группа:

406

2026

Цель работы

Изучить механизм Cross-Origin Resource Sharing, проверить ограничения политики одного источника и настроить безопасный доступ между локальными веб приложениями.

Исходная схема

Созданы три приложения: приложение 1 работает по адресу `http://localhost:8081`, приложение 2 - по адресу `http://localhost:8082`, приложение 3 - по адресу `http://localhost:8083`. Приложение 3 получает данные из базы SQLite и выдаёт таблицу `primer1` приложению 1, а таблицу `primer2` приложению 2.

Порты входят в `origin`. Поэтому страницы `localhost:8081`, `localhost:8082` и `localhost:8083` являются разными источниками, хотя работают на одном компьютере.

Ход выполнения работы

Проверка до включения CORS

Серверы были запущены в режиме без CORS. При обращении приложения 1 к API приложения 3 браузер отправил запрос и сервер вернул HTTP 200, но JavaScript не получил доступ к телу ответа. Консоль указала на отсутствие заголовка `Access-Control-Allow-Origin`. Аналогичный результат получен для приложения 2.

Запрос с нестандартным заголовком `X-Lab-Demo` вызвал предварительный OPTIONS-запрос. Без разрешающих CORS-заголовков браузер не передал ответ странице и не выполнил основной запрос. При этом обращение к данным непосредственно со страницы приложения 3 сработало, поскольку оно не пересекает `origin`.

Настройка CORS

После повторного запуска проекта с включённым CORS сервер приложения 3 стал отправлять точные разрешения: `/api/primer1` доступен только `origin http://localhost:8081`, а `/api/primer2` - только `origin http://localhost:8082`. Не использовался символ `*`.

Для `preflight` сервер разрешает только метод GET и заголовок `X-Lab-Demo`. Сервер приложения 1 отдельно разрешает приложению 2 читать ресурс `/api/info`. Передача `cookie` и заголовок `Access-Control-Allow-Credentials` не используются.

Результаты проверки

Проверка	До CORS	После CORS
Приложение 1 -> primer1	Чтение заблокировано	HTTP 200, данные primer1
Приложение 2 -> primer2	Чтение заблокировано	HTTP 200, данные primer2
Preflight OPTIONS	Нет CORS-разрешения	OPTIONS 204, затем GET 200

Чужая таблица	Данные не читаются	HTTP 403, доступ отклонён
Приложение 2 -> API приложения 1	Чтение заблокировано	HTTP 200, доступ разрешён
Режим no-cors	Непрозрачный ответ	Непрозрачный ответ, status 0

Вывод

До настройки CORS браузер блокировал чтение междоместных ответов, хотя сервер приложения 3 возвращал HTTP 200. После настройки точечных разрешений приложение 1 получило только таблицу primer1, а приложение 2 - только таблицу primer2. Предварительный запрос OPTIONS подтвердил допустимость сложного запроса до выполнения GET. Проверка чужой таблицы вернула HTTP 403. Режим no-cors не является способом обхода CORS, так как ответ остаётся непрозрачным для JavaScript.

CORS ограничивает доступ браузерного JavaScript к ответу сервера. В реальном приложении его необходимо дополнять аутентификацией и авторизацией, поскольку сам заголовок Origin не подтверждает полномочия пользователя.

Приложение А Скриншоты до включения CORS

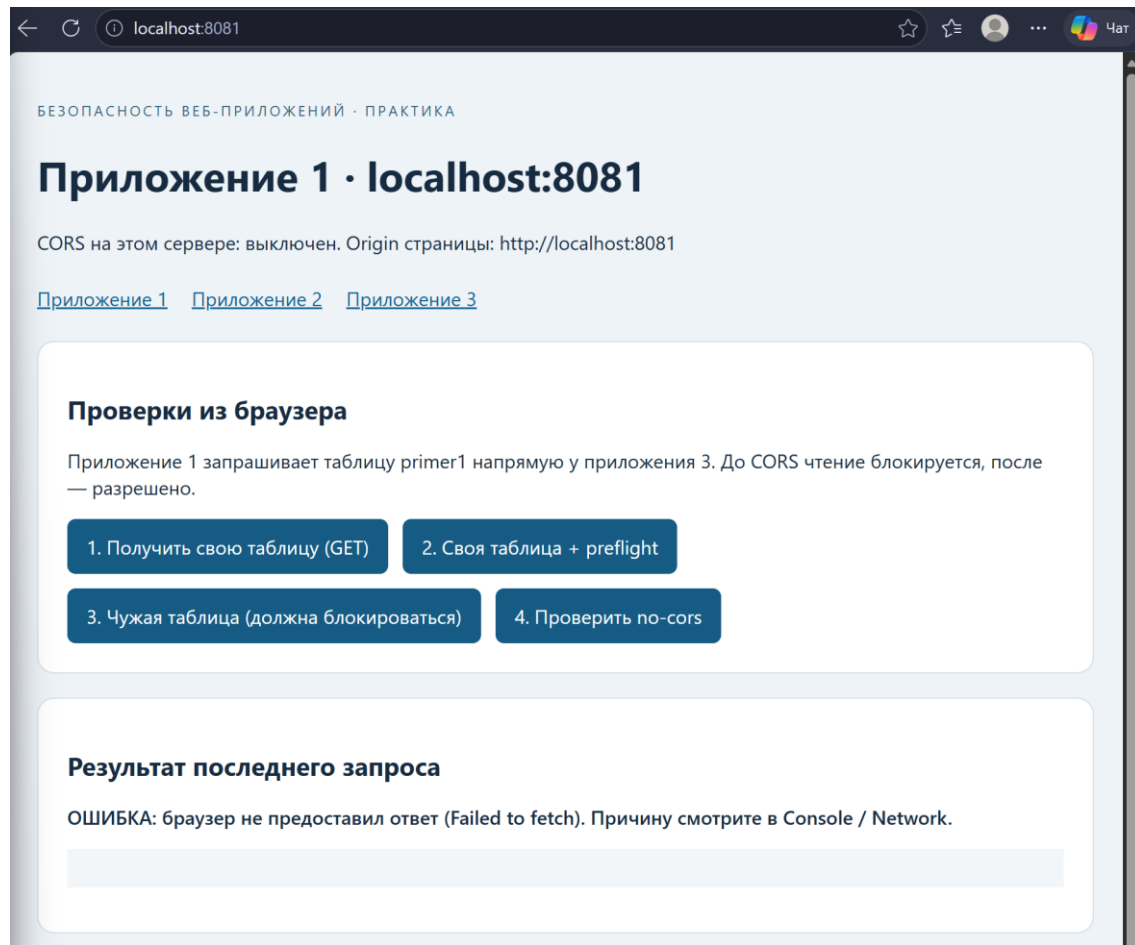


Рисунок 1 - Приложение 1 до включения CORS. Запрос к таблице primer1 завершился ошибкой чтения ответа.

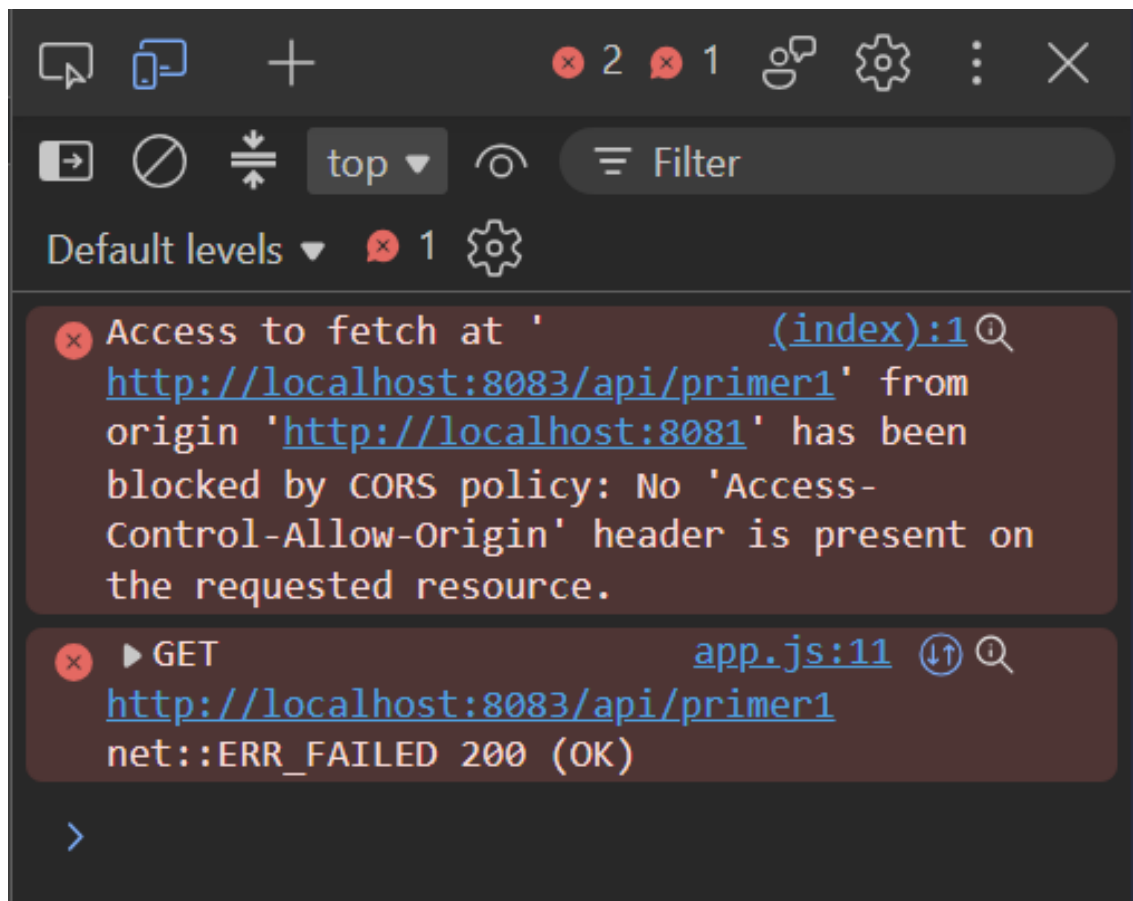


Рисунок 2 - Консоль браузера: сервер вернул ответ, но Access-Control-Allow-Origin отсутствует.

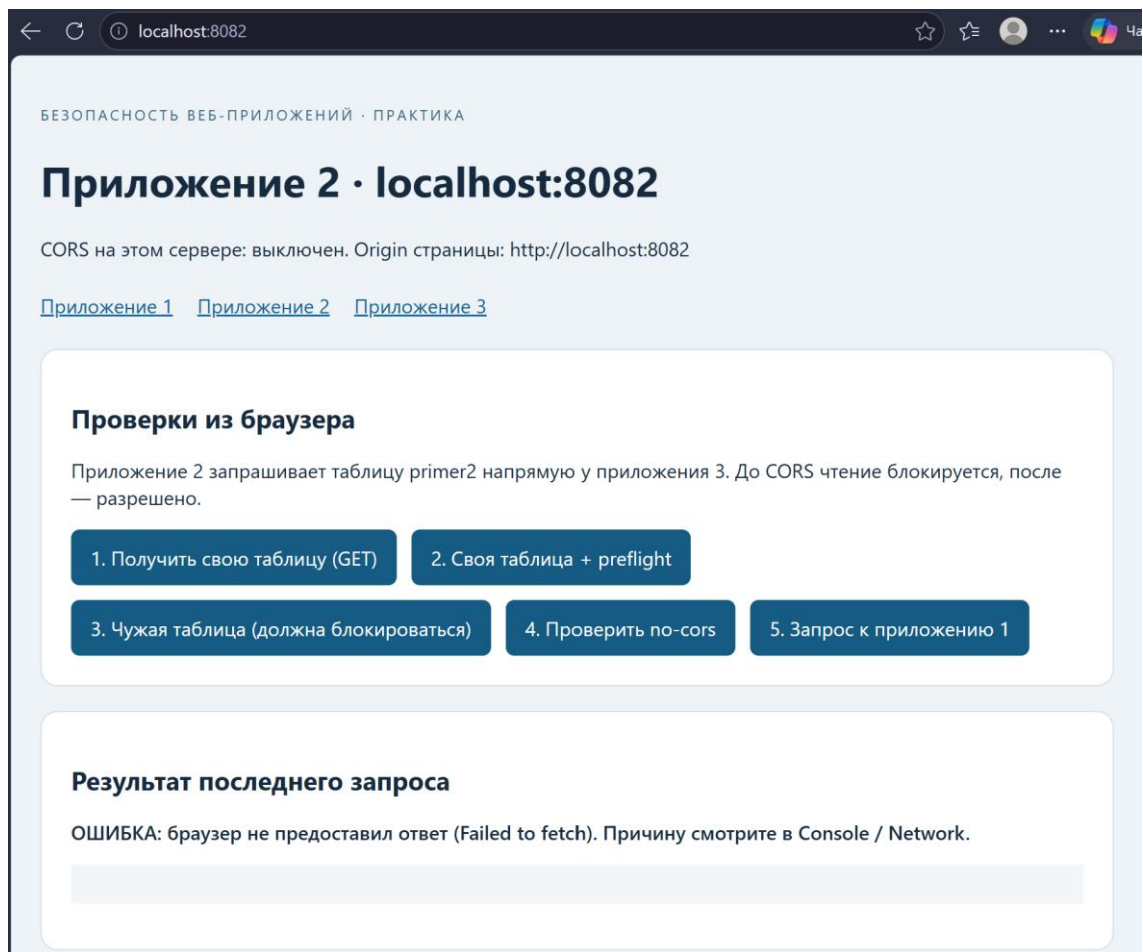


Рисунок 3 - Приложение 2 до включения CORS. Запрос таблицы primer2 заблокирован браузером.

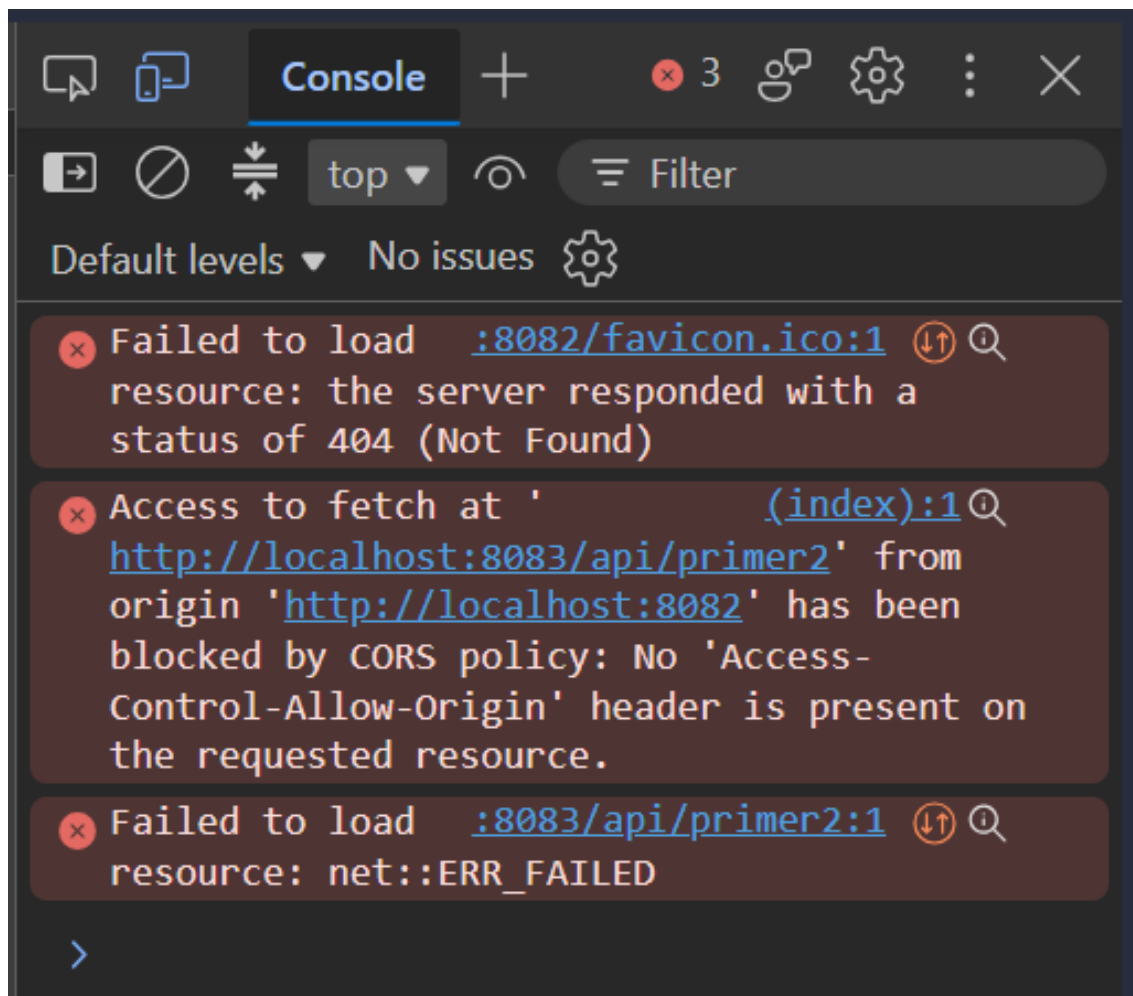


Рисунок 4 - Консоль браузера для приложения 2. Ошибка CORS при обращении к приложению 3.

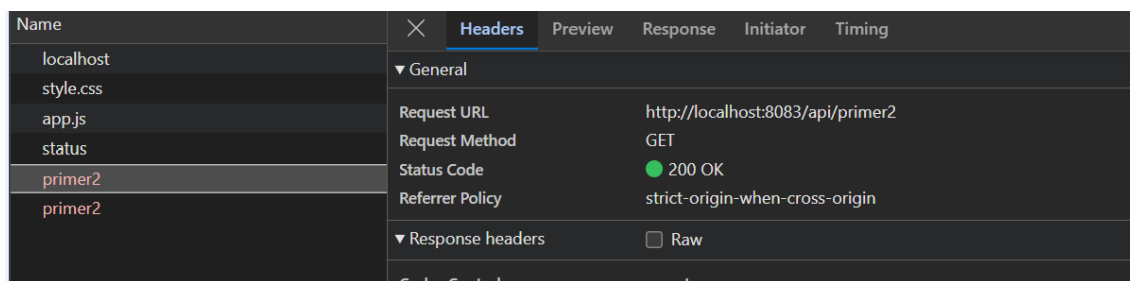


Рисунок 5 - В Network сервер приложения 3 возвращает HTTP 200 для primer2, но CORS-заголовок нет.

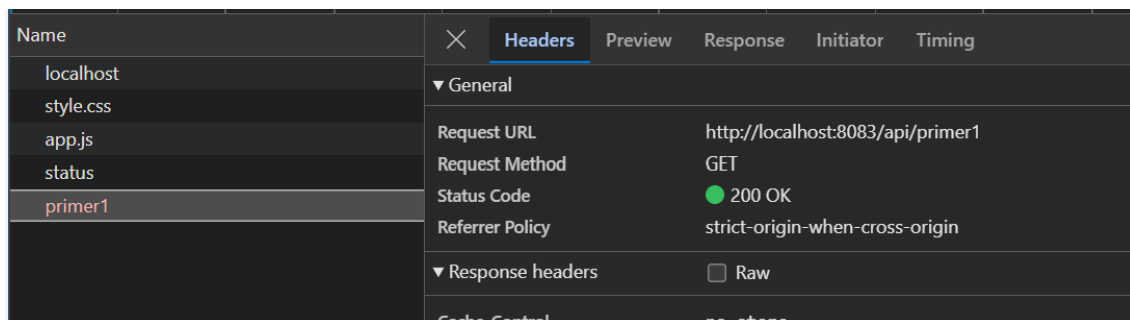


Рисунок 6 - В Network сервер приложения 3 возвращает HTTP 200 для primer1 до настройки CORS.

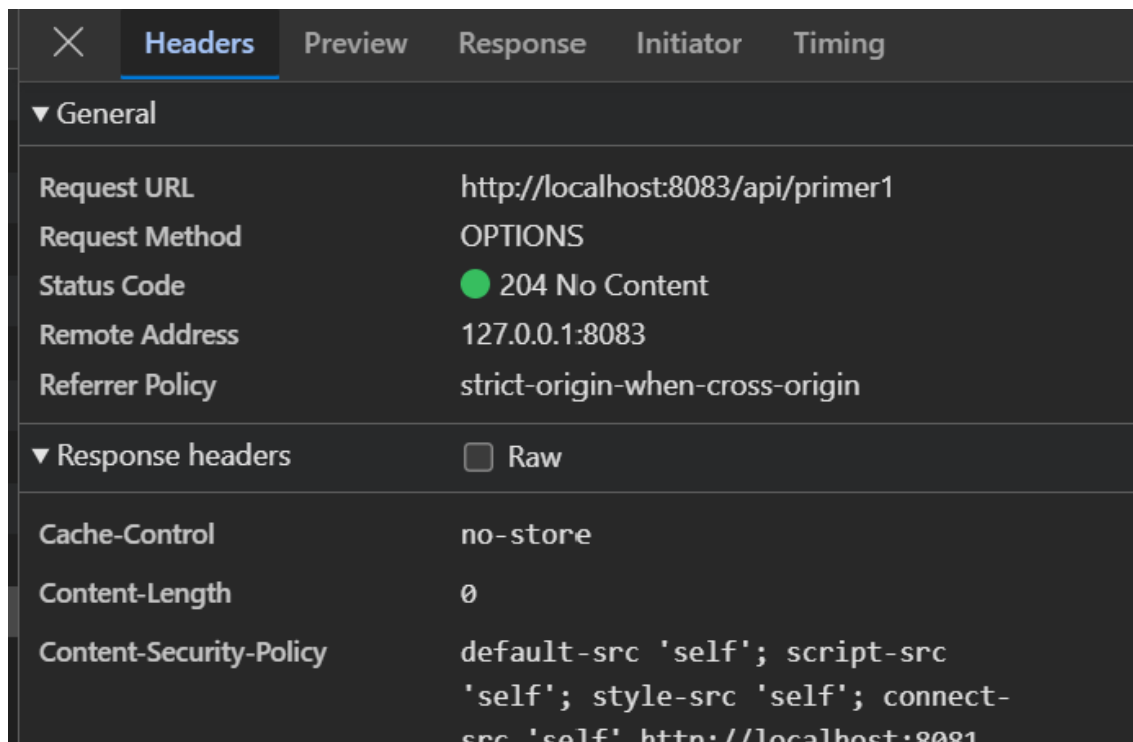


Рисунок 7 - Предварительный OPTIONS-запрос до включения CORS.

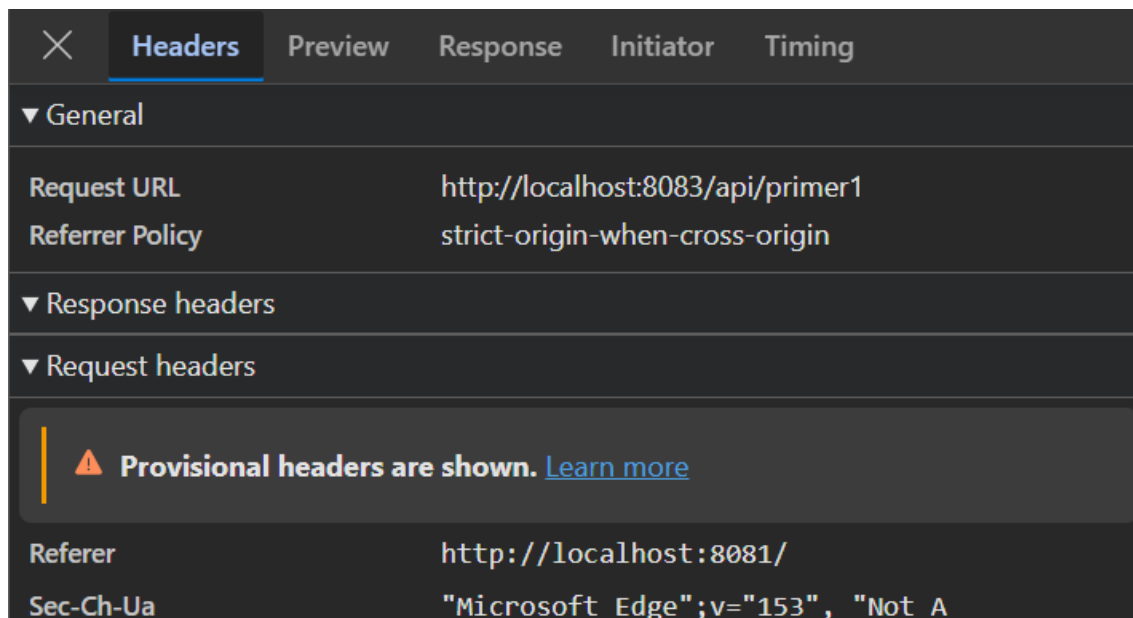


Рисунок 8 - Заголовки неуспешного запроса: браузер не предоставляет странице ответ.

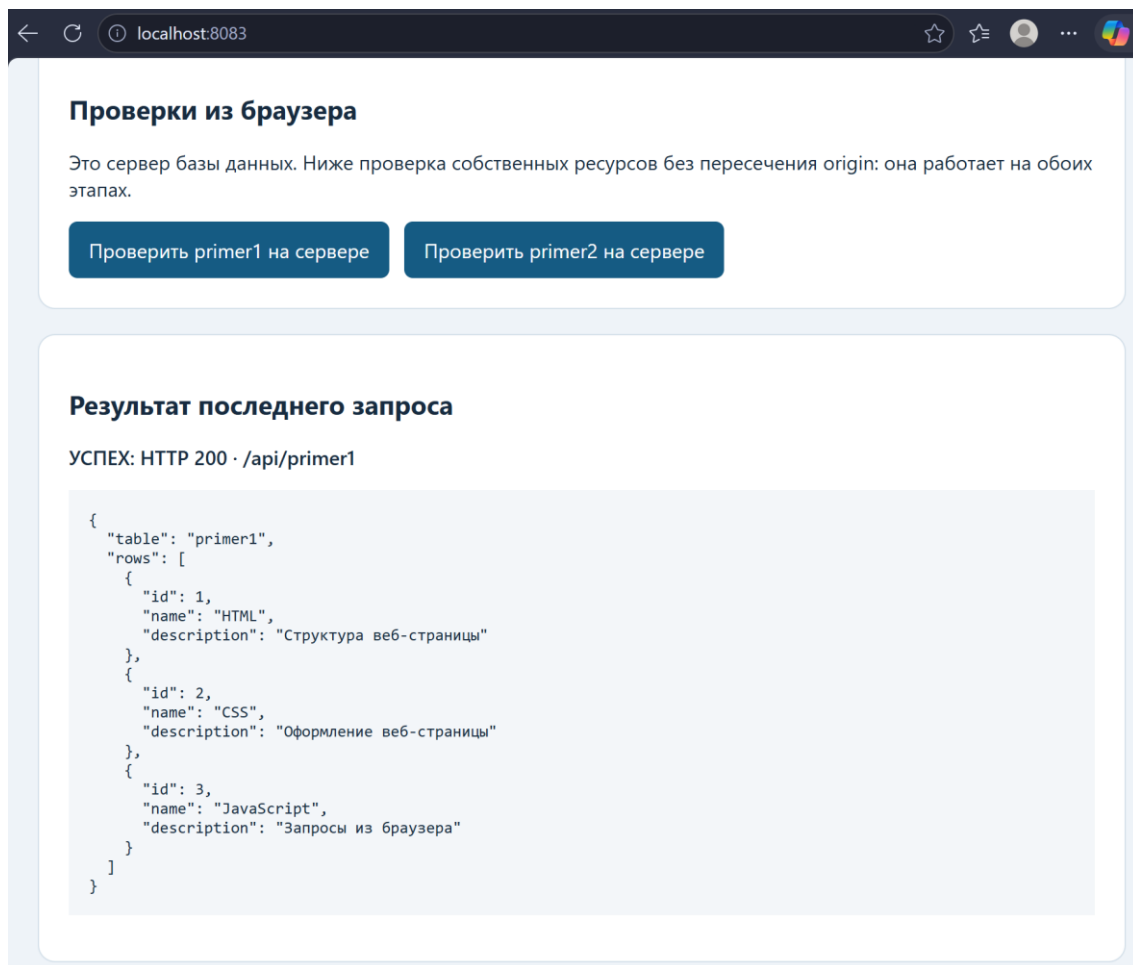


Рисунок 9 - Проверка приложения 3: таблица primer1 успешно выдана внутри origin localhost:8083.

Приложение Б Скриншоты после включения CORS

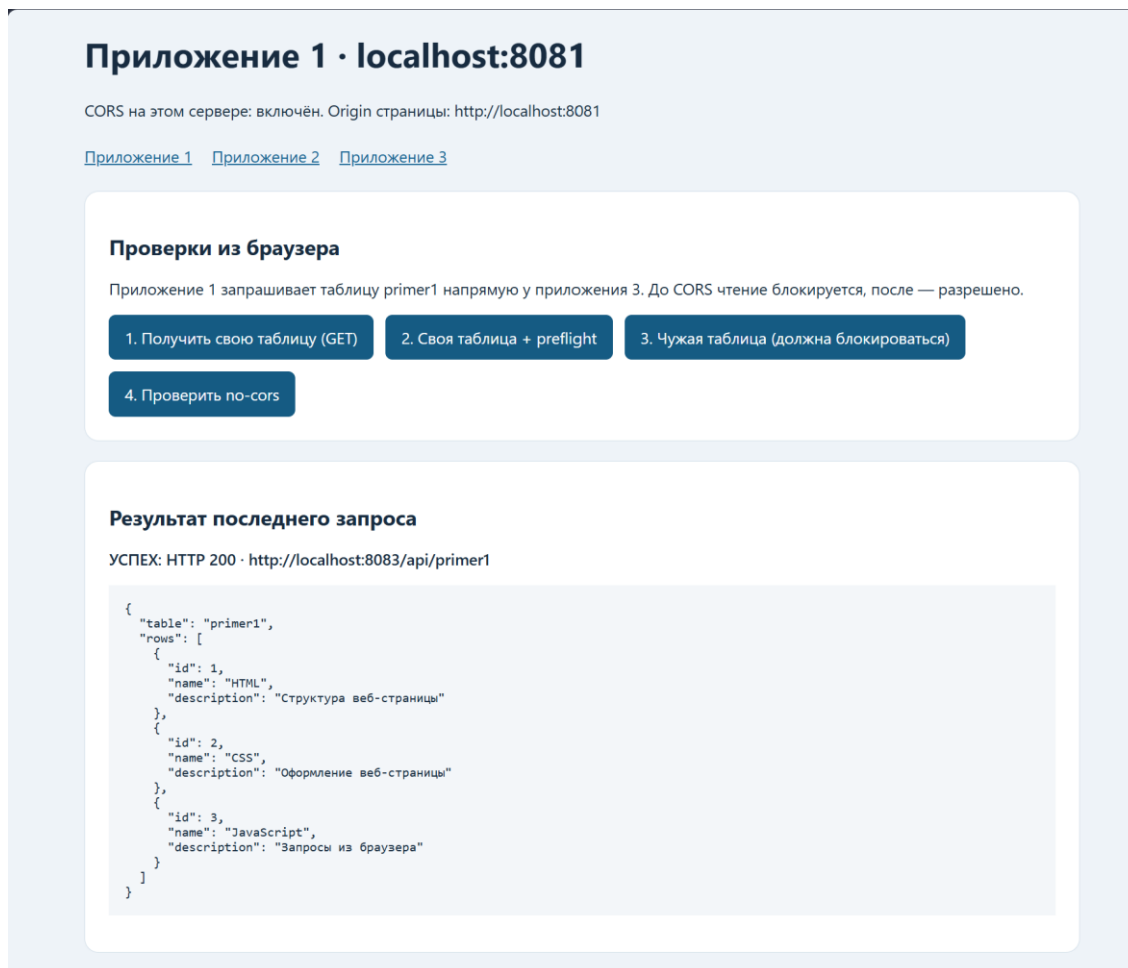


Рисунок 10 - Приложение 1 после включения CORS успешно получило таблицу primer1.

▼ Response headers	☐ Raw
Access-Control-Allow-Origin	http://localhost:8081
Cache-Control	no-store
Content-Length	295
Content-Security-Policy	default-src 'self'; script-src 'self'; style-src 'self'; connect-src 'self' http://localhost:8081 http://localhost:8083; frame-ancestors 'none'; base-uri 'none'; form-action 'none'
Content-Type	application/json; charset=utf-8
Date	Thu, 24 Sep 2026 04:37:59 GMT
Server	BaseHTTP/0.6 Python/3.14.7
Vary	Origin, Access-Control-Request-Method, Access-Control-Request-Headers
X-Content-Type-Options	nosniff

Рисунок 11 - Ответ приложения 3 содержит Access-Control-Allow-Origin: http://localhost:8081.

▼ Request headers	☐ Raw
Accept	*/*
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	ru,en;q=0.9,en-GB;q=0.8,en-US;q=0.7
Connection	keep-alive
Host	localhost:8083
Origin	http://localhost:8081
Referer	http://localhost:8081/
Sec-Ch-Ua	"Microsoft Edge";v="153", "Not_A Brand";v="8", "Chromium";v="153"
Sec-Ch-Ua-Mobile	?1
Sec-Ch-Ua-Platform	"iOS"
Sec-Fetch-Dest	empty
Sec-Fetch-Mode	cors
Sec-Fetch-Site	same-site
User-Agent	Mozilla/5.0 (iPhone; CPU iPhone OS 18_5 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.5 Mobile/15E148 Safari/604.1

Рисунок 12 - В запросе к приложению 3 указан Origin: http://localhost:8081.

Приложение 2 · localhost:8082

CORS на этом сервере: выключен. Origin страницы: http://localhost:8082

[Приложение 1](#) [Приложение 2](#) [Приложение 3](#)

Проверки из браузера

Приложение 2 запрашивает таблицу primer2 напрямую у приложения 3. До CORS чтение блокируется, после — разрешено.

1. Получить свою таблицу (GET)

2. Своя таблица + preflight

3. Чужая таблица (должна блокироваться)

4. Проверить no-cors

5. Запрос к приложению 1

Результат последнего запроса

УСПЕХ: HTTP 200 · http://localhost:8083/api/primer2

```
{
  "table": "primer2",
  "rows": [
    {
      "id": 1,
      "name": "CORS",
      "description": "Разрешения для чтения ответов из другого origin"
    },
    {
      "id": 2,
      "name": "CSP",
      "description": "Ограничение источников ресурсов страницы"
    },
    {
      "id": 3,
      "name": "SQLite",
      "description": "Локальная база данных практической работы"
    }
  ]
}
```

Рисунок 13 - Приложение 2 успешно получило таблицу primer2.

×	Headers	Preview	Response	Initiator	Timing
Request URL	http://localhost:8083/api/primer2				
Request Method	GET				
Status Code	● 200 OK				
Remote Address	127.0.0.1:8083				
Referrer Policy	strict-origin-when-cross-origin				
▼ Response headers	☐ Raw				
Access-Control-Allow-Origin	http://localhost:8082				
Cache-Control	no-store				
Content-Length	406				
Content-Security-Policy	default-src 'self'; script-src 'self'; style-src 'self'; connect-				

Рисунок 14 - Ответ приложения 3 разрешает origin `http://localhost:8082` для таблицы `primer2`.

✕	Headers	Preview	Response	Initiator	Timing
▼ General					
Request URL		http://localhost:8083/api/primer1			
Request Method		OPTIONS			
Status Code		● 204 No Content			
Remote Address		127.0.0.1:8083			
Referrer Policy		strict-origin-when-cross-origin			
▼ Response headers		☐ Raw			
Access-Control-Allow-Headers		X-Lab-Demo			
Access-Control-Allow-Methods		GET			
Access-Control-Allow-Origin		http://localhost:8081			
Access-Control-Max-Age		0			

Рисунок 15 - Preflight: OPTIONS 204 с разрешёнными GET и X-Lab-Demo.

✕	Headers	Preview	Response	Initiator	Timing
▼ General					
Request URL		http://localhost:8083/api/primer1			
Request Method		GET			
Status Code		● 200 OK			
Remote Address		127.0.0.1:8083			
Referrer Policy		strict-origin-when-cross-origin			
▼ Response headers		☐ Raw			
Access-Control-Allow-Origin		http://localhost:8081			
Cache-Control		no-store			
Content-Length		295			
Content-Security-Policy		default-src 'self'; script-src			

Рисунок 16 - Основной GET после preflight: HTTP 200 и разрешённый origin `localhost:8081`.

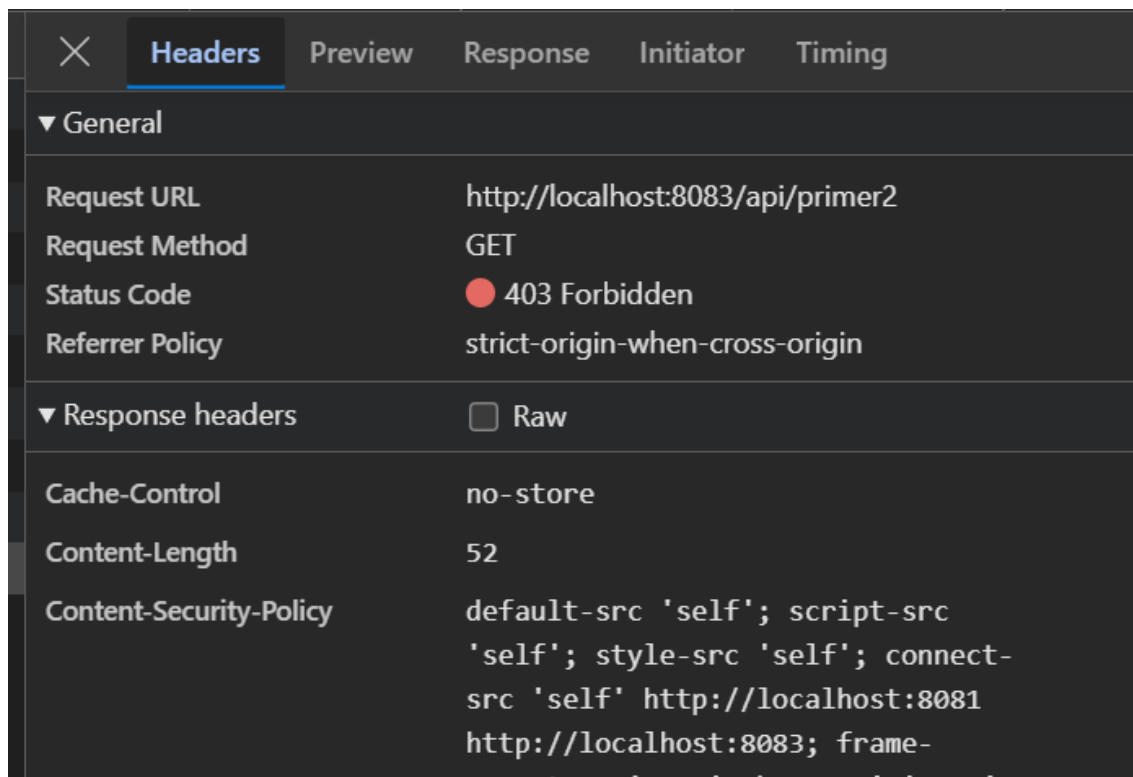


Рисунок 17 - Попытка приложения 1 получить чужую таблицу primer2 отклонена сервером: HTTP 403.

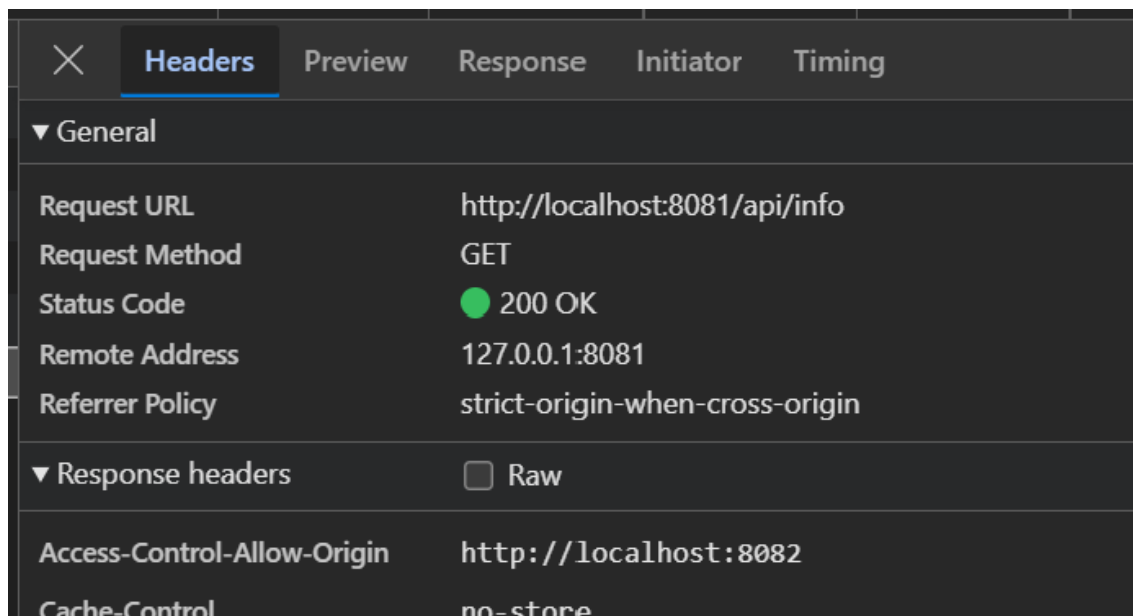


Рисунок 18 - Приложение 2 успешно получило ответ API приложения 1; разрешён origin localhost:8082.

Результат последнего запроса

OPAQUE: запрос выполнен, но JavaScript не может прочитать ответ.
`status = 0.`



Рисунок 19 - Режим `no-cors` возвращает непрозрачный ответ: JavaScript не может прочитать данные.

Результат последнего запроса

OPAQUE: запрос выполнен, но JavaScript не может прочитать ответ.
status = 0.

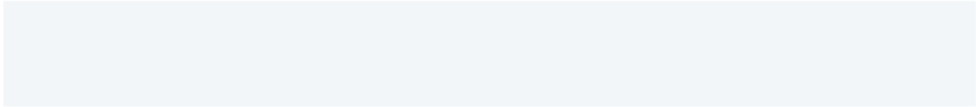


Рисунок 20 - Повторная демонстрация непрозрачного ответа no-cors со статусом 0.